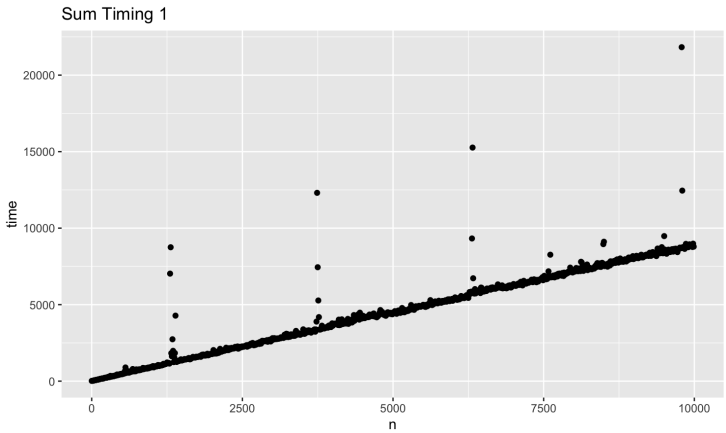
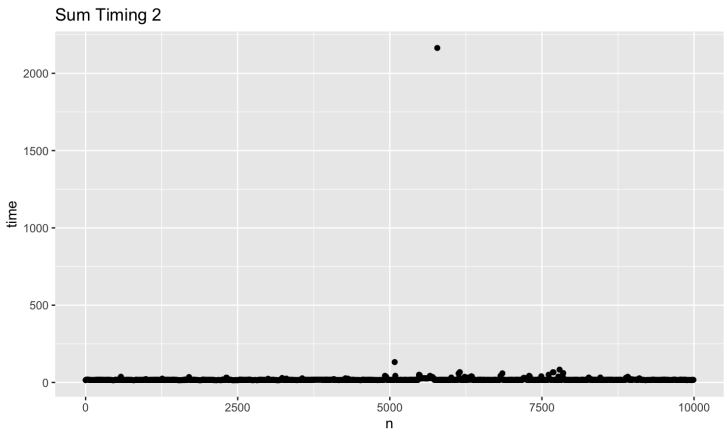
Gwynyn Hayes
Homework #1 Timing Sorting Algorithms

1. Answer the following questions.
    1.
    2. What is an algorithm?
        i. An algorithm is a set of instructions that accomplish a task. They make it so the task is repeatable and easier to do.
    3. In the context of algorithms, what does "correctness" mean? Why is it important?
        i. Correctness looks at how well an algorithm completes a task. How well does the algorithm do when repeated on different information. Does it do the same thing each time?
    4. In the context of algorithms, what does "efficiency" mean? Why is it important?
        i. Algorithms should be relatively simple so, they shouldn't use more resources then they need to or take more time then they need to. This is important because processing time can slow down an algorithm significantly and

2. Compile and run the code in sum_timing.cpp (posted on Moodle) for a timing experiment for two different functions that compute the sum of the integers 1 through n.
    1. Provide a plot of the results of your timing experiment. You may use any method to plot your result from the csv files; one way is to open the files in Excel or Google sheets, highlight the columns, and insert a scatter chart.

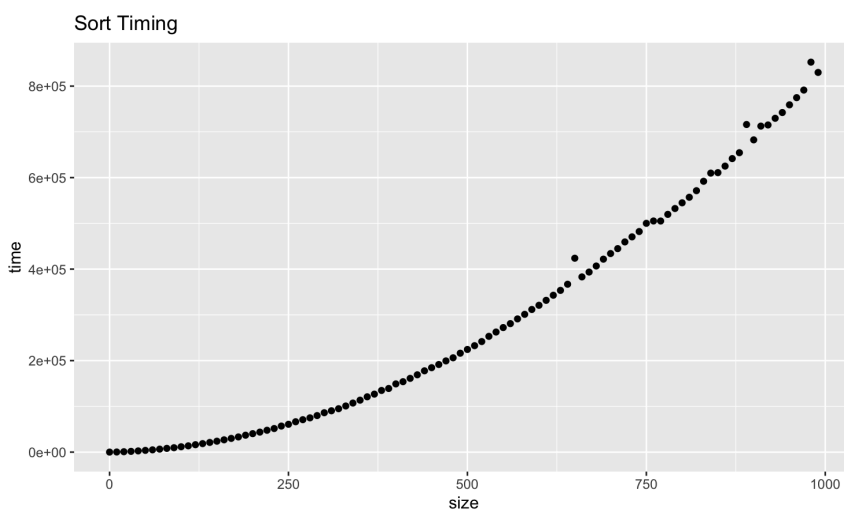




    2. Based on the results of your timing experiment, which algorithm do you think is better? Why?
        i. Based on this plot sum timing 2 is better because no matter the size of the array the algorithm takes about the same amount of time each iteration of the algorithm. Except for it seems one major outlier.

3. Looking at the code for each of the algorithms, explain how each of them works.
    i. Sum1 is looping through each number and adding each new number to the total individually
    ii. Sum2 is looking at the largest number that we want the sum of all numbers below, adding one and multiplying then dividing by 2. It's not looping through all the numbers and instead is just doing one calculation.
4. Based on a comparison of the code, why do you think one algorithm performs better than the other?
    i. I think that sum2 is the better algorithm because it is more efficient and is saving time by just computing the final sum instead of looping through every single number.

3. Compile and run the code in selection_sort.cpp (posted on Moodle), which runs a timing experiment on an implementation of selection sort (not-in-place). The program tests the correctness of the implementation by checking that it correctly sorts the test array in the function test (you'll add additional tests later). It then generates arrays with random integers, with increasing lengths of the arrays, and computes an average run time of the implementation on an array of this length. This average run time is written to a csv file, with the length of the array.
    1. Provide a plot of the results of this timing experiment. You may use any method to plot your result from the csv files; one way is to open the files in Excel or Google sheets, highlight the columns, and insert a scatter chart.

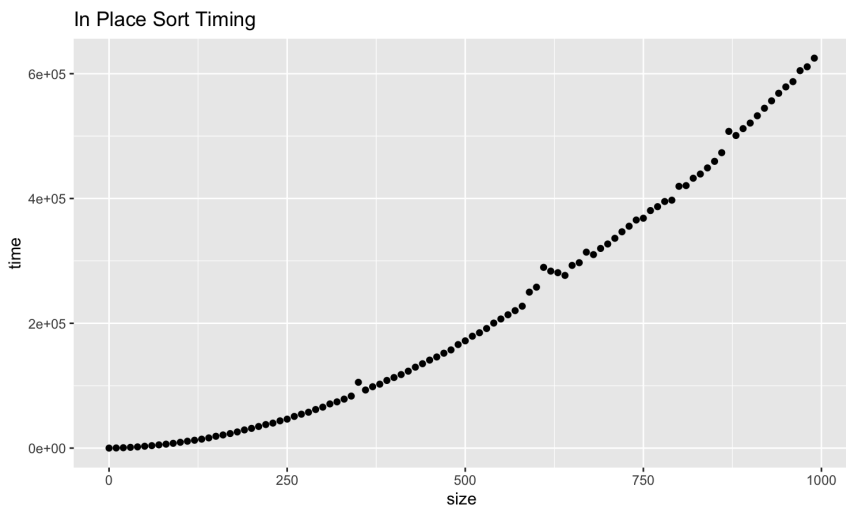


    2. Based on the results, is selection sort faster than linear, linear, or slower than linear? You can determine this by holding the edge of a sheet of paper up to the plot. If the plot curls up from the edge of the paper, the run time is slower than linear. If the plot curls down from the edge of the paper, the run time is faster than

linear. If the plot can be aligned well with the edge of the paper, then the run time is linear.

   i. The run time is slower than linear. For each increase in the size of the array the algorithm takes longer to sort the array.

4. Make a copy of the file selection_sort.cpp, and call it selection_sort_in_place.cpp.
   1. In the file selection_sort_in_place.cpp, modify the code for selection sort so that the algorithm works in place, and does not use an extra array. See the materials from class (the end of the slides from 2/11) for pseudocode for this sorting algorithm.
   2. Also in the file selection_sort_in_place.cpp, add nine additional test arrays to make a total of ten tests. Be sure to include the following in your tests: an empty array, an array of length 1, an array that is already in order, an array in reverse order, and arrays of varying lengths.
   3. Run the timing experiment for your in-place implementation of selection sort, and provide a plot of the results.



   4. Based on the results, is this version of selection sort faster than linear, linear, or slower than linear?
      i. This is slower than linear.
   5. Based on the results, is the in-place version of selection sort faster, slower, or similar in speed to the not-in-place version of selection sort? Explain your answer.
      i. The not in place version is faster because the plot has a more linear line. It also makes sense because it doesn't have to loop through each of the numbers multiple times and can just put the values into the new array already sorted.

5. (Puzzle Problem) Try to solve the following problem, and submit notes on your problem solving process (e.g., what ideas do you have? What did you try? What works? What

doesn't work? etc.). You can earn full credit by demonstrating problem solving effort, even if you do not correctly solve the puzzle. You are assembling a 1000 piece jigsaw puzzle. For each step of assembly, you connect two groups of already connected pieces. What is the smallest number of steps it could take to assemble the puzzle? Explain your reasoning.

1. The first step would be to find all pieces a match then each of the pairs finds another pair this would just divide 1000 by 2 recursively until there are no pairs left.
2. If you are always connecting one piece to another, then it would take 999 steps